Studio Proyek 1 — Fine-Tuning ANN (from scratch) pada CIFAR-10

Mata Kuliah: Pembelajaran Mendalam (IF25-40401)

Notebook ini membangun MLP/ANN standar (bukan CNN) untuk klasifikasi CIFAR-10, lalu menjalankan eksperimen sistematis terhadap optimizer, scheduler, regularisasi, dan kapasitas model. Early stopping dan checkpointing diimplementasikan manual (from scratch).

Kelompok:

- Anggota 1: Tengku Hafid Diraputra
- Anggota 2: Ahmat Prayoga Sembiring

```
# ============================================================
# SEL KOMPREHENSIF - MANDIRI
# Mencakup: semua import, seed, device, mount Google Drive
# Aman dijalankan sendirian meski runtime baru di-restart
# ============================================================

import os
import time
import copy
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split
import torchvision
import torchvision.transforms as T
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
accuracy_score

print("Semua library berhasil diimpor.")

# --- Seed untuk reproducibility ---
SEED = 42
torch.manual_seed(SEED)
np.random.seed(SEED)

# --- Cek device ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
if torch.cuda.is_available():
    print("Nama GPU:", torch.cuda.get_device_name(0))
```

```
# --- Mount Google Drive (supaya dataset & checkpoint tidak hilang saat disconnect) ---
from google.colab import drive
drive.mount('/content/drive')
```

```
Semua library berhasil diimpor.
Device: cuda
Nama GPU: Tesla T4
Mounted at /content/drive
```

1. Preparation & Preprocessing

Load CIFAR-10, terapkan augmentasi dasar pada data latih, lalu flatten setiap gambar menjadi vektor 3072 fitur (32x32x3) untuk digunakan pada MLP.

Dataset: CIFAR-10 -- 60.000 gambar berwarna 32x32 piksel dalam 10 kelas (6.000 gambar/kelas), terbagi menjadi 50.000 data training dan 10.000 data test. Sumber: https://cave.cs.toronto.edu/kriz/cifar.html. Dataset diunduh otomatis oleh `torchvision.datasets.CIFAR10` dari server resmi yang sama.

Sitasi: Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images* (Technical Report). University of Toronto.

Pada tahap ini dataset CIFAR-10 dimuat menggunakan `torchvision.datasets.CIFAR10`, lalu di-*preprocess* agar siap digunakan oleh arsitektur MLP (bukan CNN):

- **Flattening**: setiap gambar 32×32×3 diubah menjadi vektor 1D berukuran 3072 fitur, karena MLP membutuhkan input berbentuk vektor, bukan tensor 3 dimensi.
- **Augmentasi data latih**: `RandomCrop` dan `RandomHorizontalFlip` diterapkan hanya pada data training untuk menambah variasi data dan mengurangi risiko overfitting. Data validasi/test tidak diberi augmentasi agar evaluasi konsisten.
- **Normalisasi**: menggunakan mean & std resmi CIFAR-10 agar training lebih stabil.
- **Split data**: 45.000 data untuk training, 5.000 untuk validasi (diambil dari 50.000 data training asli), dan 10.000 data test terpisah.
- **Strategi cache**: dataset mentah (`.tar.gz`) disimpan permanen di Google Drive agar tidak perlu di-download ulang setiap sesi baru, sementara proses ekstraksi & pembacaan data tetap dilakukan di disk lokal Colab karena jauh lebih cepat dibanding akses langsung ke Drive.

```
CIFAR_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR_STD  = (0.2470, 0.2435, 0.2616)

train_transform = T.Compose([
    T.RandomCrop(32, padding=4),
    T.RandomHorizontalFlip(),
    T.ToTensor(),
    T.Normalize(CIFAR_MEAN, CIFAR_STD),
```

```
    T.Lambda(lambda x: torch.flatten(x)),
])

eval_transform = T.Compose([
    T.ToTensor(),
    T.Normalize(CIFAR_MEAN, CIFAR_STD),
    T.Lambda(lambda x: torch.flatten(x)),
])

# ------------------------------------------------------------------
# Strategi cache dua lapis supaya dataset TIDAK didownload/di-extract
# ulang setiap kali runtime baru / Run All lagi:
#   1. File .tar.gz asli disimpan permanen di Google Drive (persist
#      lintas sesi, sekali download saja seumur hidup).
#   2. Proses extract & training tetap di disk lokal Colab (/content),
#      karena baca/tulis di Drive (FUSE mount) jauh lebih lambat
#      dibanding disk lokal -- baik untuk extract maupun DataLoader.
# ------------------------------------------------------------------
import shutil

DRIVE_CACHE = "/content/drive/MyDrive/cifar10_data"   # penyimpanan
permanen (tar.gz saja)
LOCAL_PATH  = "/content/cifar10_data"                  # tempat kerja
aktual (cepat)
TAR_NAME    = "cifar-10-python.tar.gz"

os.makedirs(DRIVE_CACHE, exist_ok=True)
os.makedirs(LOCAL_PATH, exist_ok=True)

drive_tar = os.path.join(DRIVE_CACHE, TAR_NAME)
local_tar = os.path.join(LOCAL_PATH, TAR_NAME)

# Kalau tar sudah pernah di-cache di Drive tapi belum ada di lokal,
# salin dulu (transfer antar-server Google, hitungan detik).
if os.path.exists(drive_tar) and not os.path.exists(local_tar):
    print("Menyalin cache dataset dari Google Drive ke disk lokal...")
    shutil.copy(drive_tar, local_tar)
    print("Selesai menyalin.\n")

print(f"Menyiapkan dataset di: {LOCAL_PATH}")
print("download=True di bawah ini otomatis SKIP download kalau tar
sudah ada & valid.\n")

# download=True: torchvision hanya benar-benar mendownload dari
internet
# kalau local_tar belum ada / checksum tidak valid. Kalau sudah ada,
# langsung extract lokal (cepat, hitungan detik).
full_train = torchvision.datasets.CIFAR10(root=LOCAL_PATH, train=True,
download=True, transform=train_transform)
full_train_eval = torchvision.datasets.CIFAR10(root=LOCAL_PATH,
```

```
train=True, download=True, transform=eval_transform)
test_set = torchvision.datasets.CIFAR10(root=LOCAL_PATH, train=False,
download=True, transform=eval_transform)

# Kalau ini pertama kali download (tar baru muncul di lokal tapi belum
ada
# di Drive), simpan salinannya ke Drive supaya sesi berikutnya tidak
perlu
# download ulang dari internet sama sekali.
if os.path.exists(local_tar) and not os.path.exists(drive_tar):
    print("Menyimpan cache dataset ke Google Drive untuk sesi
berikutnya...")
    shutil.copy(local_tar, drive_tar)
    print("Cache tersimpan di Drive.\n")

CLASSES = full_train.classes
print("Dataset siap.")
print("Kelas:", CLASSES)
print("Jumlah data training:", len(full_train))
print("Jumlah data test:", len(test_set))
```

```
Menyalin cache dataset dari Google Drive ke disk lokal...
Selesai menyalin.

Menyiapkan dataset di: /content/cifar10_data
download=True di bawah ini otomatis SKIP download kalau tar sudah ada
& valid.


100%|██████████| 170M/170M [14:13<00:00, 200kB/s]

Dataset siap.
Kelas: ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
'frog', 'horse', 'ship', 'truck']
Jumlah data training: 50000
Jumlah data test: 10000
```

```
VAL_SIZE = 5000
TRAIN_SIZE = len(full_train) - VAL_SIZE

generator = torch.Generator().manual_seed(SEED)
train_idx, val_idx = random_split(range(len(full_train)), [TRAIN_SIZE,
VAL_SIZE], generator=generator)
train_idx, val_idx = train_idx.indices, val_idx.indices

train_subset = torch.utils.data.Subset(full_train, train_idx)
# validasi memakai transform tanpa augmentasi agar evaluasi konsisten
val_subset = torch.utils.data.Subset(full_train_eval, val_idx)

BATCH_SIZE = 128
```

```
train_loader = DataLoader(train_subset, batch_size=BATCH_SIZE,
shuffle=True, num_workers=2)
val_loader = DataLoader(val_subset, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2)
test_loader = DataLoader(test_set, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2)

print(f"Train: {len(train_subset)} | Val: {len(val_subset)} | Test:
{len(test_set)}")
```

```
Train: 45000 | Val: 5000 | Test: 10000
```

2. Baseline Model

Mendefinisikan kelas `ANN` yang fleksibel menggunakan `nn.Module`, dengan parameter yang dapat diatur:

- `hidden_dims`: jumlah dan ukuran hidden layer (contoh: `(512, 256)` berarti 2 hidden layer)
- `use_batchnorm`: mengaktifkan/menonaktifkan Batch Normalization
- `dropout_p`: besar probabilitas Dropout (0 = tidak aktif)

Desain fleksibel ini penting karena kelas yang sama akan dipakai ulang untuk seluruh skenario eksperimen di Bagian 4, cukup dengan mengubah parameter saat instansiasi objek.

Arsitektur baseline yang dibangun: `3072 → 512 → 256 → 10` (Input → Hidden 1 → Hidden 2 → Output), dengan total **1.707.274 parameter** yang bisa dilatih (trainable parameters).

```
class ANN(nn.Module):
    def __init__(self, input_dim=3072, hidden_dims=(512, 256),
num_classes=10,
                 use_batchnorm=False, dropout_p=0.0):
        super().__init__()
        layers = []
        in_dim = input_dim
        for h in hidden_dims:
            layers.append(nn.Linear(in_dim, h))
            if use_batchnorm:
                layers.append(nn.BatchNorm1d(h))
            layers.append(nn.ReLU())
            if dropout_p > 0:
                layers.append(nn.Dropout(dropout_p))
            in_dim = h
        layers.append(nn.Linear(in_dim, num_classes))
        self.net = nn.Sequential(*layers)

    def forward(self, x):
        return self.net(x)
```

```
baseline = ANN(hidden_dims=(512, 256))
print(baseline)
n_params = sum(p.numel() for p in baseline.parameters())
print(f"Jumlah parameter: {n_params:,}")
```

```
ANN(
  (net): Sequential(
    (0): Linear(in_features=3072, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=256, bias=True)
    (3): ReLU()
    (4): Linear(in_features=256, out_features=10, bias=True)
  )
)
Jumlah parameter: 1,707,274
```

3. Fungsi Training + Early Stopping & Checkpointing (From Scratch)

Bagian ini berisi dua fungsi inti:

`evaluate()` — menghitung loss dan akurasi model pada suatu data loader (dipakai untuk validasi maupun test), dijalankan dalam mode `eval()` dan tanpa perhitungan gradien (`torch.no_grad()`) agar lebih efisien.

`train_model()` — loop training utama yang mengimplementasikan Early Stopping dan Checkpointing secara manual (from scratch, tanpa library eksternal seperti `EarlyStopping` dari PyTorch Lightning):

- Setiap akhir epoch, model dievaluasi pada data validasi.
- Jika `val_loss` membaik (lebih kecil dari `best_val_loss` sebelumnya, dengan toleransi `1e-4`), maka bobot model disimpan menggunakan `torch.save()` sebagai checkpoint terbaik.
- Jika tidak ada perbaikan selama `patience` epoch berturut-turut (default 5), training dihentikan lebih awal (*early stopping*) untuk mencegah overfitting dan menghemat waktu komputasi.
- Setelah training selesai/berhenti, bobot terbaik yang tersimpan di-*load* kembali ke model sebelum dikembalikan, sehingga model yang dipakai selalu versi dengan validasi terbaik — bukan versi di epoch terakhir.
- Fungsi ini juga mendukung penggunaan learning rate scheduler secara opsional, termasuk scheduler yang butuh metrik sebagai input (seperti `ReduceLROnPlateau`).

```
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
```

```
            out = model(x)
            loss = criterion(out, y)
            total_loss += loss.item() * x.size(0)
            correct += (out.argmax(1) == y).sum().item()
            total += x.size(0)
    return total_loss / total, correct / total


def train_model(model, train_loader, val_loader, optimizer,
scheduler=None,
                epochs=30, patience=5,
checkpoint_path="checkpoint.pt", scheduler_needs_metric=False):
    criterion = nn.CrossEntropyLoss()
    model.to(device)

    history = {"train_loss": [], "val_loss": [], "val_acc": []}
    best_val_loss = float("inf")
    epochs_no_improve = 0
    start_time = time.time()
    stopped_epoch = epochs

    for epoch in range(1, epochs + 1):
        model.train()
        running_loss, total = 0.0, 0
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * x.size(0)
            total += x.size(0)

        train_loss = running_loss / total
        val_loss, val_acc = evaluate(model, val_loader, criterion)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        if scheduler is not None:
            if scheduler_needs_metric:
                scheduler.step(val_loss)
            else:
                scheduler.step()

        improved = val_loss < best_val_loss - 1e-4
        if improved:
            best_val_loss = val_loss
```

```
            epochs_no_improve = 0
            torch.save(model.state_dict(), checkpoint_path)
        else:
            epochs_no_improve += 1

        print(f"Epoch {epoch:02d}/{epochs} |
train_loss={train_loss:.4f} "
              f"val_loss={val_loss:.4f} val_acc={val_acc:.4f} "
              f"{'*' if improved else ''}")

        if epochs_no_improve >= patience:
            print(f"Early stopping dipicu pada epoch {epoch} (tidak
ada perbaikan {patience}x berturut-turut).")
            stopped_epoch = epoch
            break

    training_time = time.time() - start_time
    model.load_state_dict(torch.load(checkpoint_path))
    return model, history, best_val_loss, training_time, stopped_epoch
```

4. Skenario Eksperimen

Seluruh eksperimen di bawah ini menggunakan arsitektur dan data yang **sama**, kecuali satu komponen yang sedang diuji pada masing-masing skenario. Prinsip ini penting agar perbandingan antar eksperimen adil (*controlled experiment* — hanya satu variabel yang berubah setiap kali).

Fungsi `run_experiment()` dibuat sebagai wrapper agar setiap eksperimen bisa dijalankan secara konsisten dan hasilnya (model, history, waktu training, epoch berhenti, dsb.) otomatis tersimpan ke dalam dictionary `results` untuk memudahkan perbandingan dan visualisasi di Bagian 5.

```
results = {}
EPOCHS = 25
PATIENCE = 5

def run_experiment(name, hidden_dims=(512, 256), use_batchnorm=False,
dropout_p=0.0,
                   optimizer_name="adam", lr=1e-3, momentum=0.9,
                   scheduler_name=None):
    torch.manual_seed(SEED)
    model = ANN(hidden_dims=hidden_dims, use_batchnorm=use_batchnorm,
dropout_p=dropout_p)

    if optimizer_name == "adam":
        optimizer = optim.Adam(model.parameters(), lr=lr)
    elif optimizer_name == "sgd_momentum":
        optimizer = optim.SGD(model.parameters(), lr=lr,
momentum=momentum)
```

```
    else:
        raise ValueError(optimizer_name)

    scheduler, needs_metric = None, False
    if scheduler_name == "steplr":
        scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=8,
gamma=0.5)
    elif scheduler_name == "plateau":
        scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode="min", factor=0.5, patience=2)
        needs_metric = True

    print(f"\n=== Eksperimen: {name} ===")
    model, history, best_val_loss, train_time, stopped_epoch =
train_model(
        model, train_loader, val_loader, optimizer, scheduler,
        epochs=EPOCHS, patience=PATIENCE,
        checkpoint_path=f"ckpt_{name}.pt",
        scheduler_needs_metric=needs_metric,
    )

    results[name] = {
        "model": model,
        "history": history,
        "best_val_loss": best_val_loss,
        "best_val_acc": max(history["val_acc"]),
        "train_time": train_time,
        "stopped_epoch": stopped_epoch,
        "config": dict(hidden_dims=hidden_dims,
use_batchnorm=use_batchnorm, dropout_p=dropout_p,
                       optimizer_name=optimizer_name, lr=lr,
scheduler_name=scheduler_name),
    }
    return model, history
```

4a. Baseline

Eksperimen kontrol/pembanding utama. Menggunakan arsitektur dasar (512, 256), optimizer **Adam** (lr=1e-3), **tanpa** scheduler, **tanpa** Dropout, dan **tanpa** Batch Normalization.

Hasil eksperimen ini menjadi acuan (baseline) untuk menilai apakah perubahan pada eksperimen selanjutnya (optimizer, regularisasi, kapasitas) benar-benar memberikan peningkatan performa atau tidak.

Hasil: Early stopping terpicu di epoch 11, dengan validation accuracy terbaik **46,92%**.

```
run_experiment("baseline", hidden_dims=(512, 256),
optimizer_name="adam", scheduler_name=None)
```

```
=== Eksperimen: baseline ===
Epoch 01/25 | train_loss=1.8068 val_loss=1.7485 val_acc=0.3948 *
Epoch 02/25 | train_loss=1.6379 val_loss=1.6502 val_acc=0.4136 *
Epoch 03/25 | train_loss=1.5688 val_loss=1.6467 val_acc=0.4096 *
Epoch 04/25 | train_loss=1.5299 val_loss=1.5933 val_acc=0.4320 *
Epoch 05/25 | train_loss=1.5014 val_loss=1.5750 val_acc=0.4334 *
Epoch 06/25 | train_loss=1.4733 val_loss=1.5151 val_acc=0.4632 *
Epoch 07/25 | train_loss=1.4557 val_loss=1.5485 val_acc=0.4564
Epoch 08/25 | train_loss=1.4354 val_loss=1.5727 val_acc=0.4452
Epoch 09/25 | train_loss=1.4177 val_loss=1.5344 val_acc=0.4692
Epoch 10/25 | train_loss=1.4054 val_loss=1.5721 val_acc=0.4540
Epoch 11/25 | train_loss=1.3918 val_loss=1.5609 val_acc=0.4506
Early stopping dipicu pada epoch 11 (tidak ada perbaikan 5x berturut-
turut).

(ANN(
   (net): Sequential(
     (0): Linear(in_features=3072, out_features=512, bias=True)
     (1): ReLU()
     (2): Linear(in_features=512, out_features=256, bias=True)
     (3): ReLU()
     (4): Linear(in_features=256, out_features=10, bias=True)
   )
 ),
 {'train_loss': [1.8067818906148274,
   1.6379091743469238,
   1.5687518948872885,
   1.5298772321065266,
   1.501369137234158,
   1.4733279411527846,
   1.4556701517952813,
   1.4354332824918958,
   1.417746386994256,
   1.4053956981447009,
   1.391787825902303],
  'val_loss': [1.7484613708496093,
   1.6501628831863404,
   1.6466684844970703,
   1.593324022293091,
   1.5750179830551148,
   1.5151244644165038,
   1.5485122838974,
   1.5726916313171386,
   1.5344074214935304,
   1.572106354522705,
   1.5609303966522217],
  'val_acc': [0.3948,
   0.4136,
   0.4096,
```

```
   0.432,
   0.4334,
   0.4632,
   0.4564,
   0.4452,
   0.4692,
   0.454,
   0.4506]})
```

4b. Optimizer & Scheduler

Menguji 3 skenario untuk melihat pengaruh pemilihan optimizer dan scheduler terhadap kecepatan konvergensi dan hasil akhir:

1. **`optim_sgd_momentum`** — mengganti Adam dengan SGD + Momentum (lr=1e-2, momentum=0.9), untuk membandingkan karakteristik konvergensi kedua optimizer.
2. **`optim_adam_steplr`** — Adam dikombinasikan dengan scheduler **StepLR** (menurunkan learning rate 50% setiap 8 epoch).
3. **`optim_adam_plateau`** — Adam dikombinasikan dengan scheduler **ReduceLROnPlateau** (menurunkan learning rate 50% jika validation loss tidak membaik selama 2 epoch).

Tujuannya adalah mencari kombinasi optimizer + scheduler yang menghasilkan konvergensi paling stabil dan akurasi validasi tertinggi, untuk kemudian dipakai sebagai dasar pada eksperimen regularisasi dan kapasitas berikutnya.

Hasil: SGD+Momentum (51,72%) > Adam+ReduceLROnPlateau (52,98%) ≈ Adam+StepLR (52,02%), semuanya mengungguli baseline Adam tanpa scheduler (46,92%).

```
run_experiment("optim_sgd_momentum", hidden_dims=(512, 256),
optimizer_name="sgd_momentum", lr=1e-2, momentum=0.9)
run_experiment("optim_adam_steplr", hidden_dims=(512, 256),
optimizer_name="adam", scheduler_name="steplr")
run_experiment("optim_adam_plateau", hidden_dims=(512, 256),
optimizer_name="adam", scheduler_name="plateau")


=== Eksperimen: optim_sgd_momentum ===
Epoch 01/25 | train_loss=1.8556 val_loss=1.7315 val_acc=0.3858 *
Epoch 02/25 | train_loss=1.6617 val_loss=1.6148 val_acc=0.4322 *
Epoch 03/25 | train_loss=1.5823 val_loss=1.5755 val_acc=0.4436 *
Epoch 04/25 | train_loss=1.5326 val_loss=1.5287 val_acc=0.4642 *
Epoch 05/25 | train_loss=1.5008 val_loss=1.5094 val_acc=0.4726 *
Epoch 06/25 | train_loss=1.4724 val_loss=1.4484 val_acc=0.4906 *
Epoch 07/25 | train_loss=1.4486 val_loss=1.4601 val_acc=0.4872
Epoch 08/25 | train_loss=1.4243 val_loss=1.4705 val_acc=0.4844
Epoch 09/25 | train_loss=1.4021 val_loss=1.4444 val_acc=0.4954 *
Epoch 10/25 | train_loss=1.3857 val_loss=1.4471 val_acc=0.4862
Epoch 11/25 | train_loss=1.3722 val_loss=1.4345 val_acc=0.5052 *
```

```
Epoch 12/25 | train_loss=1.3577 val_loss=1.3963 val_acc=0.5080 *
Epoch 13/25 | train_loss=1.3467 val_loss=1.3852 val_acc=0.5124 *
Epoch 14/25 | train_loss=1.3358 val_loss=1.3802 val_acc=0.5172 *
Epoch 15/25 | train_loss=1.3206 val_loss=1.4056 val_acc=0.5080
Epoch 16/25 | train_loss=1.3134 val_loss=1.3892 val_acc=0.5118
Epoch 17/25 | train_loss=1.3050 val_loss=1.4033 val_acc=0.5038
Epoch 18/25 | train_loss=1.2937 val_loss=1.4032 val_acc=0.5072
Epoch 19/25 | train_loss=1.2909 val_loss=1.3804 val_acc=0.5062
Early stopping dipicu pada epoch 19 (tidak ada perbaikan 5x berturut-
turut).

=== Eksperimen: optim_adam_steplr ===
Epoch 01/25 | train_loss=1.8068 val_loss=1.7485 val_acc=0.3948 *
Epoch 02/25 | train_loss=1.6379 val_loss=1.6502 val_acc=0.4136 *
Epoch 03/25 | train_loss=1.5688 val_loss=1.6467 val_acc=0.4096 *
Epoch 04/25 | train_loss=1.5299 val_loss=1.5933 val_acc=0.4320 *
Epoch 05/25 | train_loss=1.5014 val_loss=1.5750 val_acc=0.4334 *
Epoch 06/25 | train_loss=1.4733 val_loss=1.5151 val_acc=0.4632 *
Epoch 07/25 | train_loss=1.4557 val_loss=1.5485 val_acc=0.4564
Epoch 08/25 | train_loss=1.4354 val_loss=1.5727 val_acc=0.4452
Epoch 09/25 | train_loss=1.3681 val_loss=1.4709 val_acc=0.4808 *
Epoch 10/25 | train_loss=1.3434 val_loss=1.4888 val_acc=0.4854
Epoch 11/25 | train_loss=1.3319 val_loss=1.4567 val_acc=0.4868 *
Epoch 12/25 | train_loss=1.3189 val_loss=1.4406 val_acc=0.4976 *
Epoch 13/25 | train_loss=1.3112 val_loss=1.4208 val_acc=0.5050 *
Epoch 14/25 | train_loss=1.2992 val_loss=1.4193 val_acc=0.5022 *
Epoch 15/25 | train_loss=1.2881 val_loss=1.4501 val_acc=0.4966
Epoch 16/25 | train_loss=1.2823 val_loss=1.4515 val_acc=0.4894
Epoch 17/25 | train_loss=1.2469 val_loss=1.4127 val_acc=0.5074 *
Epoch 18/25 | train_loss=1.2307 val_loss=1.4106 val_acc=0.5074 *
Epoch 19/25 | train_loss=1.2289 val_loss=1.3799 val_acc=0.5150 *
Epoch 20/25 | train_loss=1.2173 val_loss=1.3982 val_acc=0.5142
Epoch 21/25 | train_loss=1.2127 val_loss=1.3960 val_acc=0.5154
Epoch 22/25 | train_loss=1.2035 val_loss=1.3994 val_acc=0.5172
Epoch 23/25 | train_loss=1.1991 val_loss=1.3815 val_acc=0.5202
Epoch 24/25 | train_loss=1.1970 val_loss=1.3818 val_acc=0.5156
Early stopping dipicu pada epoch 24 (tidak ada perbaikan 5x berturut-
turut).

=== Eksperimen: optim_adam_plateau ===
Epoch 01/25 | train_loss=1.8068 val_loss=1.7485 val_acc=0.3948 *
Epoch 02/25 | train_loss=1.6379 val_loss=1.6502 val_acc=0.4136 *
Epoch 03/25 | train_loss=1.5688 val_loss=1.6467 val_acc=0.4096 *
Epoch 04/25 | train_loss=1.5299 val_loss=1.5933 val_acc=0.4320 *
Epoch 05/25 | train_loss=1.5014 val_loss=1.5750 val_acc=0.4334 *
Epoch 06/25 | train_loss=1.4733 val_loss=1.5151 val_acc=0.4632 *
Epoch 07/25 | train_loss=1.4557 val_loss=1.5485 val_acc=0.4564
Epoch 08/25 | train_loss=1.4354 val_loss=1.5727 val_acc=0.4452
Epoch 09/25 | train_loss=1.4177 val_loss=1.5344 val_acc=0.4692
```

```
Epoch 10/25 | train_loss=1.3541 val_loss=1.4911 val_acc=0.4762 *
Epoch 11/25 | train_loss=1.3309 val_loss=1.4525 val_acc=0.4866 *
Epoch 12/25 | train_loss=1.3188 val_loss=1.4469 val_acc=0.4966 *
Epoch 13/25 | train_loss=1.3113 val_loss=1.4292 val_acc=0.4930 *
Epoch 14/25 | train_loss=1.2992 val_loss=1.4193 val_acc=0.5024 *
Epoch 15/25 | train_loss=1.2884 val_loss=1.4406 val_acc=0.4992
Epoch 16/25 | train_loss=1.2809 val_loss=1.4217 val_acc=0.5036
Epoch 17/25 | train_loss=1.2743 val_loss=1.4221 val_acc=0.5004
Epoch 18/25 | train_loss=1.2388 val_loss=1.4117 val_acc=0.5088 *
Epoch 19/25 | train_loss=1.2267 val_loss=1.3844 val_acc=0.5192 *
Epoch 20/25 | train_loss=1.2173 val_loss=1.3919 val_acc=0.5182
Epoch 21/25 | train_loss=1.2127 val_loss=1.4128 val_acc=0.5092
Epoch 22/25 | train_loss=1.2042 val_loss=1.4126 val_acc=0.5106
Epoch 23/25 | train_loss=1.1846 val_loss=1.3722 val_acc=0.5256 *
Epoch 24/25 | train_loss=1.1793 val_loss=1.3691 val_acc=0.5280 *
Epoch 25/25 | train_loss=1.1787 val_loss=1.3541 val_acc=0.5298 *

(ANN(
   (net): Sequential(
     (0): Linear(in_features=3072, out_features=512, bias=True)
     (1): ReLU()
     (2): Linear(in_features=512, out_features=256, bias=True)
     (3): ReLU()
     (4): Linear(in_features=256, out_features=10, bias=True)
   )
 ),
 {'train_loss': [1.8067818906148274,
   1.6379091743469238,
   1.5687518948872885,
   1.5298772321065266,
   1.501369137234158,
   1.4733279411527846,
   1.4556701517952813,
   1.4354332824918958,
   1.417746386994256,
   1.3541045874701605,
   1.3309254426744248,
   1.318783546617296,
   1.3112897898779976,
   1.299211787668864,
   1.2883813120947945,
   1.280913657591078,
   1.2742671714782714,
   1.238812370300293,
   1.2267267841974894,
   1.2172621603859795,
   1.2127371679094103,
   1.2041758401234945,
   1.184599637667338,
   1.179272861650255,
```

```
   1.178658802710639],
  'val_loss': [1.7484613708496093,
   1.6501628831863404,
   1.6466684844970703,
   1.593324022293091,
   1.5750179830551148,
   1.5151244644165038,
   1.5485122838974,
   1.5726916313171386,
   1.5344074214935304,
   1.4910520435333252,
   1.4524722766876221,
   1.4468567657470703,
   1.4292180446624756,
   1.4193136127471924,
   1.4406488912582398,
   1.4217134311676025,
   1.4221274864196778,
   1.4116727615356446,
   1.3843665088653565,
   1.391919287109375,
   1.412847674560547,
   1.4125625673294067,
   1.3722337152481079,
   1.3691259708404542,
   1.3541050521850586],
  'val_acc': [0.3948,
   0.4136,
   0.4096,
   0.432,
   0.4334,
   0.4632,
   0.4564,
   0.4452,
   0.4692,
   0.4762,
   0.4866,
   0.4966,
   0.493,
   0.5024,
   0.4992,
   0.5036,
   0.5004,
   0.5088,
   0.5192,
   0.5182,
   0.5092,
   0.5106,
   0.5256,
```

```
   0.528,
   0.5298]})
```

4c. Regularisasi (Dropout & Batch Normalization)

Menguji pengaruh dua teknik regularisasi yang umum dipakai untuk mencegah overfitting pada arsitektur baseline:

1. **`reg_dropout`** — menambahkan Dropout dengan probabilitas 0.3 di setiap hidden layer.
2. **`reg_batchnorm`** — menambahkan Batch Normalization di setiap hidden layer (tanpa Dropout).
3. **`reg_dropout_batchnorm`** — kombinasi keduanya (Dropout 0.3 + Batch Normalization) sekaligus.

Eksperimen ini penting untuk melihat apakah kedua teknik regularisasi benar-benar membantu generalisasi model, atau justru terlalu agresif hingga menyebabkan *underfitting*.

Hasil: BatchNorm saja (58,68%) jauh mengungguli semua kombinasi lainnya. Dropout saja justru menjadi hasil **terburuk** dari seluruh eksperimen (43,14%) — mengindikasikan Dropout p=0.3 terlalu agresif untuk ukuran model ini.

```
run_experiment("reg_dropout", hidden_dims=(512, 256), dropout_p=0.3,
optimizer_name="adam")
run_experiment("reg_batchnorm", hidden_dims=(512, 256),
use_batchnorm=True, optimizer_name="adam")
run_experiment("reg_dropout_batchnorm", hidden_dims=(512, 256),
use_batchnorm=True, dropout_p=0.3, optimizer_name="adam")
```

```
=== Eksperimen: reg_dropout ===
Epoch 01/25 | train_loss=1.9133 val_loss=1.7830 val_acc=0.3582 *
Epoch 02/25 | train_loss=1.7882 val_loss=1.7094 val_acc=0.3834 *
Epoch 03/25 | train_loss=1.7485 val_loss=1.7282 val_acc=0.3796
Epoch 04/25 | train_loss=1.7127 val_loss=1.7170 val_acc=0.3776
Epoch 05/25 | train_loss=1.6906 val_loss=1.6935 val_acc=0.3988 *
Epoch 06/25 | train_loss=1.6779 val_loss=1.6963 val_acc=0.4074
Epoch 07/25 | train_loss=1.6635 val_loss=1.6791 val_acc=0.4060 *
Epoch 08/25 | train_loss=1.6532 val_loss=1.6718 val_acc=0.4186 *
Epoch 09/25 | train_loss=1.6387 val_loss=1.6825 val_acc=0.4140
Epoch 10/25 | train_loss=1.6331 val_loss=1.6451 val_acc=0.4066 *
Epoch 11/25 | train_loss=1.6232 val_loss=1.6485 val_acc=0.4264
Epoch 12/25 | train_loss=1.6163 val_loss=1.6182 val_acc=0.4258 *
Epoch 13/25 | train_loss=1.6096 val_loss=1.6294 val_acc=0.4194
Epoch 14/25 | train_loss=1.6061 val_loss=1.6383 val_acc=0.4282
Epoch 15/25 | train_loss=1.5996 val_loss=1.6316 val_acc=0.4312
Epoch 16/25 | train_loss=1.5971 val_loss=1.6517 val_acc=0.4046
Epoch 17/25 | train_loss=1.5961 val_loss=1.6211 val_acc=0.4314
Early stopping dipicu pada epoch 17 (tidak ada perbaikan 5x berturut-
turut).
```

```
=== Eksperimen: reg_batchnorm ===
Epoch 01/25 | train_loss=1.7719 val_loss=1.6101 val_acc=0.4284 *
Epoch 02/25 | train_loss=1.5953 val_loss=1.4818 val_acc=0.4754 *
Epoch 03/25 | train_loss=1.5257 val_loss=1.4322 val_acc=0.4870 *
Epoch 04/25 | train_loss=1.4816 val_loss=1.4103 val_acc=0.4978 *
Epoch 05/25 | train_loss=1.4479 val_loss=1.3722 val_acc=0.5114 *
Epoch 06/25 | train_loss=1.4231 val_loss=1.3242 val_acc=0.5294 *
Epoch 07/25 | train_loss=1.4007 val_loss=1.3305 val_acc=0.5282
Epoch 08/25 | train_loss=1.3765 val_loss=1.3326 val_acc=0.5218
Epoch 09/25 | train_loss=1.3571 val_loss=1.2858 val_acc=0.5436 *
Epoch 10/25 | train_loss=1.3432 val_loss=1.2897 val_acc=0.5406
Epoch 11/25 | train_loss=1.3249 val_loss=1.2430 val_acc=0.5612 *
Epoch 12/25 | train_loss=1.3133 val_loss=1.2565 val_acc=0.5516
Epoch 13/25 | train_loss=1.3007 val_loss=1.2343 val_acc=0.5628 *
Epoch 14/25 | train_loss=1.2885 val_loss=1.2424 val_acc=0.5592
Epoch 15/25 | train_loss=1.2722 val_loss=1.2225 val_acc=0.5636 *
Epoch 16/25 | train_loss=1.2651 val_loss=1.2159 val_acc=0.5732 *
Epoch 17/25 | train_loss=1.2542 val_loss=1.2238 val_acc=0.5702
Epoch 18/25 | train_loss=1.2450 val_loss=1.2078 val_acc=0.5714 *
Epoch 19/25 | train_loss=1.2413 val_loss=1.1893 val_acc=0.5790 *
Epoch 20/25 | train_loss=1.2257 val_loss=1.2007 val_acc=0.5740
Epoch 21/25 | train_loss=1.2158 val_loss=1.1922 val_acc=0.5732
Epoch 22/25 | train_loss=1.2079 val_loss=1.1858 val_acc=0.5784 *
Epoch 23/25 | train_loss=1.1965 val_loss=1.1807 val_acc=0.5786 *
Epoch 24/25 | train_loss=1.1926 val_loss=1.1676 val_acc=0.5854 *
Epoch 25/25 | train_loss=1.1866 val_loss=1.1615 val_acc=0.5868 *

=== Eksperimen: reg_dropout_batchnorm ===
Epoch 01/25 | train_loss=1.8573 val_loss=1.6548 val_acc=0.4124 *
Epoch 02/25 | train_loss=1.7040 val_loss=1.5456 val_acc=0.4450 *
Epoch 03/25 | train_loss=1.6457 val_loss=1.5008 val_acc=0.4618 *
Epoch 04/25 | train_loss=1.6041 val_loss=1.4576 val_acc=0.4804 *
Epoch 05/25 | train_loss=1.5772 val_loss=1.4288 val_acc=0.4902 *
Epoch 06/25 | train_loss=1.5565 val_loss=1.3829 val_acc=0.5094 *
Epoch 07/25 | train_loss=1.5347 val_loss=1.4036 val_acc=0.5006
Epoch 08/25 | train_loss=1.5200 val_loss=1.3920 val_acc=0.4960
Epoch 09/25 | train_loss=1.5038 val_loss=1.3493 val_acc=0.5118 *
Epoch 10/25 | train_loss=1.4931 val_loss=1.3405 val_acc=0.5158 *
Epoch 11/25 | train_loss=1.4800 val_loss=1.3287 val_acc=0.5264 *
Epoch 12/25 | train_loss=1.4690 val_loss=1.3213 val_acc=0.5324 *
Epoch 13/25 | train_loss=1.4647 val_loss=1.3115 val_acc=0.5342 *
Epoch 14/25 | train_loss=1.4550 val_loss=1.3069 val_acc=0.5364 *
Epoch 15/25 | train_loss=1.4405 val_loss=1.2898 val_acc=0.5476 *
Epoch 16/25 | train_loss=1.4372 val_loss=1.2831 val_acc=0.5440 *
Epoch 17/25 | train_loss=1.4350 val_loss=1.2886 val_acc=0.5440
Epoch 18/25 | train_loss=1.4266 val_loss=1.2777 val_acc=0.5448 *
Epoch 19/25 | train_loss=1.4227 val_loss=1.2639 val_acc=0.5490 *
Epoch 20/25 | train_loss=1.4118 val_loss=1.2651 val_acc=0.5456
```

```
Epoch 21/25 | train_loss=1.4012 val_loss=1.2620 val_acc=0.5454 *
Epoch 22/25 | train_loss=1.3982 val_loss=1.2529 val_acc=0.5534 *
Epoch 23/25 | train_loss=1.3945 val_loss=1.2473 val_acc=0.5580 *
Epoch 24/25 | train_loss=1.3943 val_loss=1.2443 val_acc=0.5496 *
Epoch 25/25 | train_loss=1.3874 val_loss=1.2287 val_acc=0.5630 *

(ANN(
   (net): Sequential(
     (0): Linear(in_features=3072, out_features=512, bias=True)
     (1): BatchNorm1d(512, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
     (2): ReLU()
     (3): Dropout(p=0.3, inplace=False)
     (4): Linear(in_features=512, out_features=256, bias=True)
     (5): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True,
track_running_stats=True)
     (6): ReLU()
     (7): Dropout(p=0.3, inplace=False)
     (8): Linear(in_features=256, out_features=10, bias=True)
   )
 ),
 {'train_loss': [1.8572973006778293,
   1.7039844575670031,
   1.6457156388812595,
   1.6040793354034424,
   1.5772428548812867,
   1.5564574829525417,
   1.5346858089023165,
   1.5199551015218098,
   1.5037858831617568,
   1.4931336866166856,
   1.4799877616670396,
   1.4689759761386447,
   1.4646806628757052,
   1.4550254750145806,
   1.4405118154101901,
   1.4371586043251885,
   1.4350356509950426,
   1.4265882704628838,
   1.4227458685557048,
   1.4118194238026938,
   1.4012324069341024,
   1.398178012826707 6,
   1.3944814795388116,
   1.3942892666922675,
   1.3874115929285684],
  'val_loss': [1.6547989166259767,
   1.5455988683700561,
   1.500847211074829,
   1.4575948455810548,
```

```
   1.4288154517173768,
   1.382929359817505,
   1.403613042449951,
   1.3920421020507812,
   1.3493217353820801,
   1.3405130655288697,
   1.328710965538025,
   1.321333650970459,
   1.311468985748291,
   1.3068765628814698,
   1.2898048372268678,
   1.2830558280944824,
   1.2886475622177125,
   1.2777349590301514,
   1.2639272207260133,
   1.2650897441864013,
   1.2620474628448486,
   1.2529290651321412,
   1.2473467391967774,
   1.2442918348312377,
   1.2286944324493407],
  'val_acc': [0.4124,
   0.445,
   0.4618,
   0.4804,
   0.4902,
   0.5094,
   0.5006,
   0.496,
   0.5118,
   0.5158,
   0.5264,
   0.5324,
   0.5342,
   0.5364,
   0.5476,
   0.544,
   0.544,
   0.5448,
   0.549,
   0.5456,
   0.5454,
   0.5534,
   0.558,
   0.5496,
   0.563]})
```

4d. Kapasitas Model (Jumlah Hidden Layer / Neuron)

Menguji pengaruh ukuran (kapasitas) arsitektur terhadap akurasi dan waktu training:

1. **`capacity_shallow`** — model lebih kecil, hanya 1 hidden layer (256 neuron).
2. **`capacity_deep`** — model lebih besar, 4 hidden layer (1024 → 512 → 256 → 128 neuron).

Tujuan eksperimen ini adalah mengevaluasi *trade-off* antara ukuran model, akurasi yang dicapai, dan beban waktu komputasi — apakah menambah kapasitas model secara otomatis meningkatkan performa atau tidak.

Hasil: Model shallow (1 layer) tampil lebih buruk dari baseline (43,36%). Model deep (4 layer) sedikit lebih baik dari baseline (49,44%) namun tetap kalah jauh dari `reg_batchnorm`, sambil memakan waktu training lebih lama.

```
run_experiment("capacity_shallow", hidden_dims=(256,),
optimizer_name="adam")
run_experiment("capacity_deep", hidden_dims=(1024, 512, 256, 128),
optimizer_name="adam")


=== Eksperimen: capacity_shallow ===
Epoch 01/25 | train_loss=1.8439 val_loss=1.8261 val_acc=0.3914 *
Epoch 02/25 | train_loss=1.6824 val_loss=1.7630 val_acc=0.4120 *
Epoch 03/25 | train_loss=1.6355 val_loss=1.7750 val_acc=0.4186
Epoch 04/25 | train_loss=1.6008 val_loss=1.9999 val_acc=0.3930
Epoch 05/25 | train_loss=1.5861 val_loss=1.7866 val_acc=0.4004
Epoch 06/25 | train_loss=1.5602 val_loss=1.9892 val_acc=0.3952
Epoch 07/25 | train_loss=1.5445 val_loss=1.7487 val_acc=0.4336 *
Epoch 08/25 | train_loss=1.5394 val_loss=1.8972 val_acc=0.4060
Epoch 09/25 | train_loss=1.5243 val_loss=1.9427 val_acc=0.4294
Epoch 10/25 | train_loss=1.5152 val_loss=1.7984 val_acc=0.4294
Epoch 11/25 | train_loss=1.5138 val_loss=1.8159 val_acc=0.4304
Epoch 12/25 | train_loss=1.5028 val_loss=2.2431 val_acc=0.3980
Early stopping dipicu pada epoch 12 (tidak ada perbaikan 5x berturut-
turut).

=== Eksperimen: capacity_deep ===
Epoch 01/25 | train_loss=1.8420 val_loss=1.8125 val_acc=0.3468 *
Epoch 02/25 | train_loss=1.6621 val_loss=1.6648 val_acc=0.4202 *
Epoch 03/25 | train_loss=1.5715 val_loss=1.6654 val_acc=0.4190
Epoch 04/25 | train_loss=1.5270 val_loss=1.5724 val_acc=0.4486 *
Epoch 05/25 | train_loss=1.4782 val_loss=1.5600 val_acc=0.4458 *
Epoch 06/25 | train_loss=1.4536 val_loss=1.5662 val_acc=0.4502
Epoch 07/25 | train_loss=1.4298 val_loss=1.5120 val_acc=0.4612 *
Epoch 08/25 | train_loss=1.4042 val_loss=1.5005 val_acc=0.4658 *
Epoch 09/25 | train_loss=1.3850 val_loss=1.4998 val_acc=0.4680 *
Epoch 10/25 | train_loss=1.3666 val_loss=1.4505 val_acc=0.4944 *
Epoch 11/25 | train_loss=1.3453 val_loss=1.4836 val_acc=0.4712
Epoch 12/25 | train_loss=1.3323 val_loss=1.4397 val_acc=0.4928 *
```

```
Epoch 13/25 | train_loss=1.3211 val_loss=1.4954 val_acc=0.4752
Epoch 14/25 | train_loss=1.3048 val_loss=1.5145 val_acc=0.4696
Epoch 15/25 | train_loss=1.2953 val_loss=1.4808 val_acc=0.4786
Epoch 16/25 | train_loss=1.2865 val_loss=1.4889 val_acc=0.4686
Epoch 17/25 | train_loss=1.2725 val_loss=1.5198 val_acc=0.4810
Early stopping dipicu pada epoch 17 (tidak ada perbaikan 5x berturut-turut).

(ANN(
   (net): Sequential(
     (0): Linear(in_features=3072, out_features=1024, bias=True)
     (1): ReLU()
     (2): Linear(in_features=1024, out_features=512, bias=True)
     (3): ReLU()
     (4): Linear(in_features=512, out_features=256, bias=True)
     (5): ReLU()
     (6): Linear(in_features=256, out_features=128, bias=True)
     (7): ReLU()
     (8): Linear(in_features=128, out_features=10, bias=True)
   )
 ),
 {'train_loss': [1.842000116051568,
   1.6620572200351291,
   1.5715162942674425,
   1.5269758223639593,
   1.4781727222230698,
   1.4536381663428413,
   1.4298478392071194,
   1.4042409887313843,
   1.3849807488759358,
   1.3666070271174113,
   1.3453479356977676,
   1.3323198359171549,
   1.3211293912039863,
   1.3048268904156155,
   1.2953024800830417,
   1.2865154473834568,
   1.2724950099521213],
  'val_loss': [1.8124895774841308,
   1.664776245880127,
   1.6653717014312743,
   1.5724366521835327,
   1.5600041604995727,
   1.5661709297180175,
   1.5120100395202636,
   1.5005176378250122,
   1.4998180097579956,
   1.450455124282837,
   1.4836410717010498,
   1.4396901538848876,
```

```
   1.4954440580368042,
   1.5144908535003663,
   1.4808296110153198,
   1.4888503717422485,
   1.519821585083008],
  'val_acc': [0.3468,
   0.4202,
   0.419,
   0.4486,
   0.4458,
   0.4502,
   0.4612,
   0.4658,
   0.468,
   0.4944,
   0.4712,
   0.4928,
   0.4752,
   0.4696,
   0.4786,
   0.4686,
   0.481]})
```

5. Evaluasi Akhir

Seluruh hasil dari 9 eksperimen dirangkum ke dalam tabel `summary_df` (berisi validation accuracy, validation loss, waktu training, dan epoch saat early stopping terpicu), lalu diurutkan berdasarkan **validation accuracy** tertinggi untuk menentukan model terbaik secara otomatis.

Visualisasi yang ditampilkan:

- **Kurva Train Loss vs Validation Loss** untuk seluruh eksperimen, guna membandingkan kecepatan konvergensi dan mendeteksi gejala overfitting/underfitting.
- **Bar chart perbandingan validation accuracy** antar eksperimen untuk memudahkan identifikasi model terbaik secara visual.
- **Evaluasi akhir pada test set** menggunakan model dengan validation accuracy tertinggi (`reg_batchnorm`), guna memastikan performa model tetap konsisten pada data yang belum pernah dilihat sama sekali.
- **Confusion Matrix** untuk menganalisis kelas mana saja yang paling sering tertukar (misclassified) oleh model.

```
summary_rows = []
for name, r in results.items():
    summary_rows.append({
        "experiment": name,
        "best_val_acc": r["best_val_acc"],
        "best_val_loss": r["best_val_loss"],
        "train_time_sec": r["train_time"],
        "stopped_epoch": r["stopped_epoch"],
```

```
    })

import pandas as pd
summary_df = pd.DataFrame(summary_rows).sort_values("best_val_acc",
ascending=False).reset_index(drop=True)
summary_df
```


```
{"summary":"{\n  \"name\": \"summary_df\",\n  \"rows\": 9,\n
\"fields\": [\n    {\n      \"column\": \"experiment\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 9,\n        \"samples\": [\n
\"capacity_shallow\",\n          \"reg_dropout_batchnorm\",\n
\"capacity_deep\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"best_val_acc\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.05357126095211871,\n        \"min\":
0.4314,\n        \"max\": 0.5868,\n        \"num_unique_values\": 9,\n
\"samples\": [\n          0.4336,\n          0.563,\n          0.4944\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"best_val_loss\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.18287549255472585,\n        \"min\":
1.1614589115142822,\n        \"max\": 1.7487407945632936,\n
\"num_unique_values\": 9,\n        \"samples\": [\n
1.7487407945632936,\n          1.2286944324493407,\n
1.4396901538848876\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"train_time_sec\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 102.77371374439576,\n        \"min\":
204.92185735702515,\n        \"max\": 463.7023708820343,\n
\"num_unique_values\": 9,\n        \"samples\": [\n
218.94913911819458,\n          460.956356048584,\n
313.572505235672\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"stopped_epoch\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 5,\n        \"min\": 11,\n
\"max\": 25,\n        \"num_unique_values\": 6,\n        \"samples\":
[\n          25,\n          24,\n          12\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"summary_df"}
```

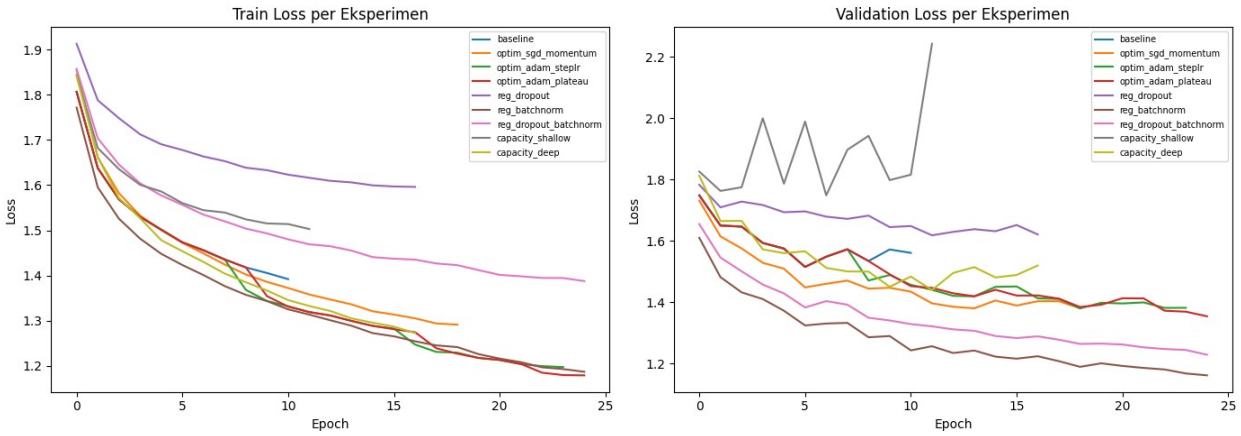

```
best_name = summary_df.iloc[0]["experiment"]
best_model = results[best_name]["model"]
print("Eksperimen terbaik berdasarkan validation accuracy:",
best_name)
```

```
Eksperimen terbaik berdasarkan validation accuracy: reg_batchnorm
```

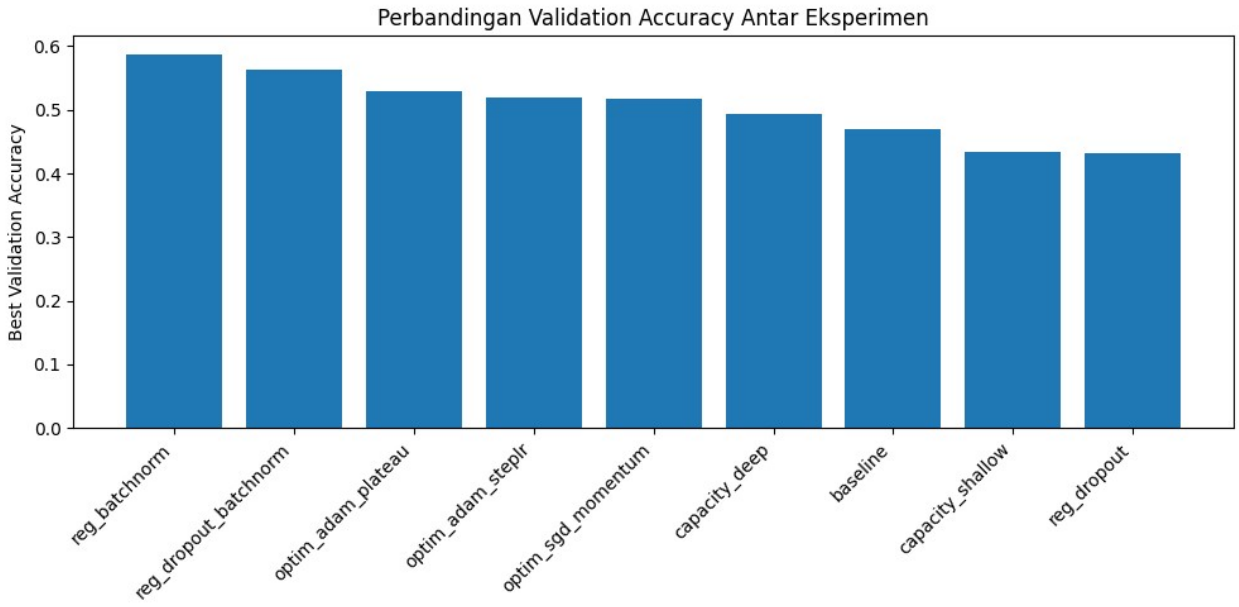
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
for name, r in results.items():
```

```
    axes[0].plot(r["history"]["train_loss"], label=name)
    axes[1].plot(r["history"]["val_loss"], label=name)

axes[0].set_title("Train Loss per Eksperimen")
axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss")
axes[1].set_title("Validation Loss per Eksperimen")
axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Loss")
for ax in axes:
    ax.legend(fontsize=7)
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(10, 5))
plt.bar(summary_df["experiment"], summary_df["best_val_acc"])
plt.xticks(rotation=45, ha="right")
plt.ylabel("Best Validation Accuracy")
plt.title("Perbandingan Validation Accuracy Antar Eksperimen")
plt.tight_layout()
plt.show()
```

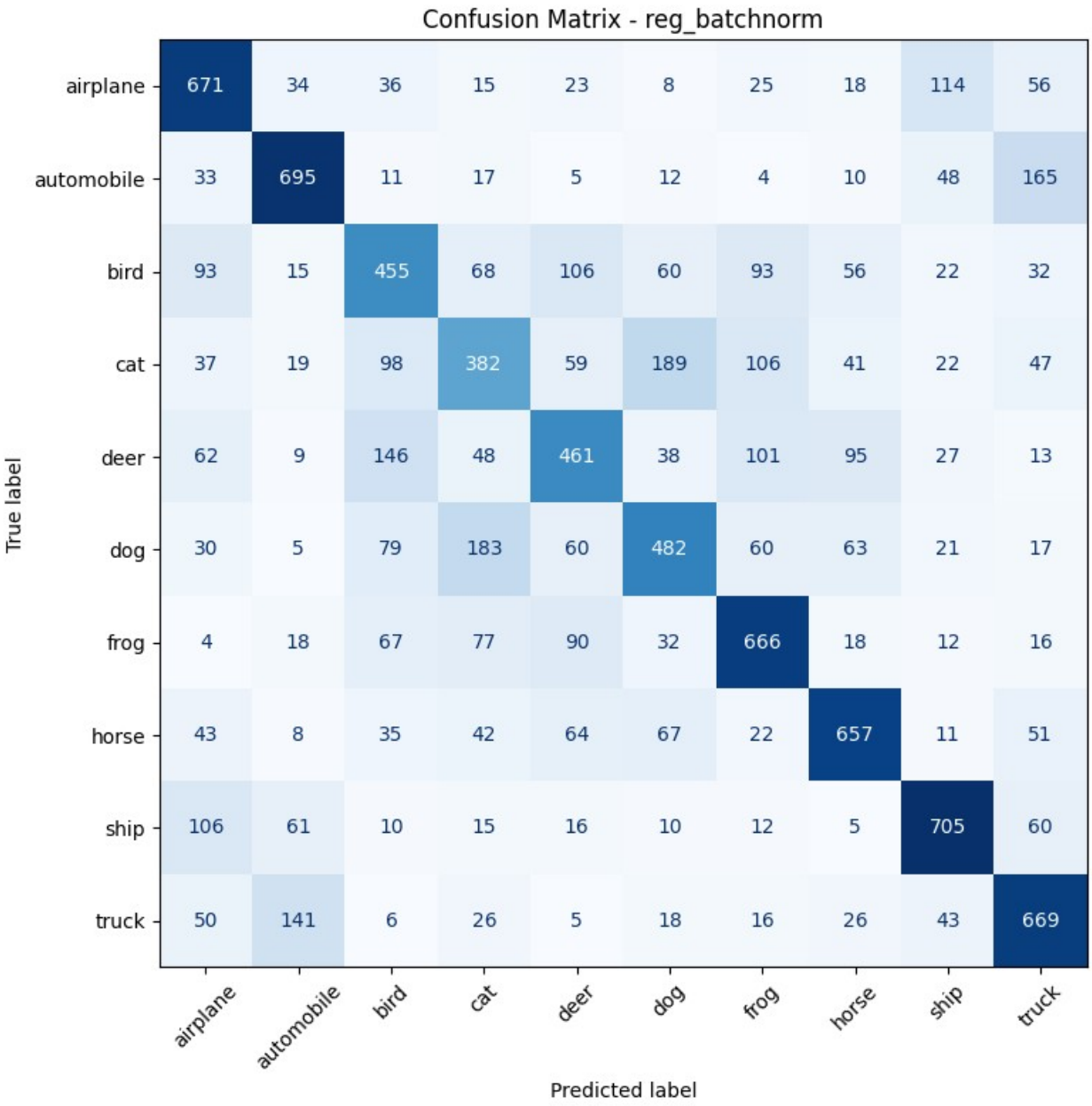



```
criterion = nn.CrossEntropyLoss()
test_loss, test_acc = evaluate(best_model, test_loader, criterion)
print(f"Model terbaik: {best_name}")
print(f"Test Loss: {test_loss:.4f} | Test Accuracy: {test_acc:.4f}")

best_model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for x, y in test_loader:
        x = x.to(device)
        out = best_model(x)
        preds = out.argmax(1).cpu().numpy()
        all_preds.extend(preds)
        all_labels.extend(y.numpy())

cm = confusion_matrix(all_labels, all_preds)
fig, ax = plt.subplots(figsize=(8, 8))
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=CLASSES)
disp.plot(ax=ax, xticks_rotation=45, cmap="Blues", colorbar=False)
plt.title(f"Confusion Matrix - {best_name}")
plt.tight_layout()
plt.show()

Model terbaik: reg_batchnorm
Test Loss: 1.1581 | Test Accuracy: 0.5843
```



6. Kesimpulan

Model terbaik: `reg_batchnorm` Arsitektur baseline (3072 → 512 → 256 → 10) + Batch Normalization, tanpa Dropout, optimizer Adam default tanpa scheduler tambahan.

- Validation Accuracy: **58,68%**
- Test Accuracy: **58,43%**
- Test Loss: **1,1581**
- Gap validation–test hanya **0,25%** → model tidak overfitting, generalisasi konsisten antara data validasi dan data test.

Analisis per Skenario Eksperimen

1. Optimizer (Adam vs SGD+Momentum) SGD+Momentum (val_acc 51,72%) mengungguli Adam baseline polos (val_acc 46,92%), namun membutuhkan lebih banyak epoch sebelum early stopping (19 vs 11 epoch). Adam konvergen lebih cepat di awal training, tetapi berhenti membaik lebih dini.

2. Scheduler Baik StepLR (val_acc 52,02%) maupun ReduceLROnPlateau (val_acc 52,98%) terbukti memperbaiki hasil dibanding Adam tanpa scheduler (46,92%). Penurunan learning rate secara bertahap membantu model terus membaik di epoch-epoch akhir tanpa overshoot.

3. Regularisasi (Dropout & Batch Normalization) Hasilnya kontras tajam:

- BatchNorm sendirian (`reg_batchnorm`) mencapai val_acc **58,68%** — **terbaik** dari seluruh eksperimen.
- Dropout sendirian (`reg_dropout`) hanya mencapai val_acc **43,14%** — **terburuk** dari semua eksperimen, bahkan di bawah baseline.
- Kombinasi Dropout+BatchNorm (56,30%) lebih baik dari Dropout saja, tetapi tetap kalah dari BatchNorm saja.

Kesimpulan: BatchNorm sangat efektif menstabilkan training pada arsitektur ini, sedangkan Dropout p=0.3 terlalu agresif untuk model MLP berukuran sedang ini sehingga menyebabkan *underfitting*, bukan mencegah overfitting.

4. Kapasitas Model Model kecil (`capacity_shallow`, 1 hidden layer) mencapai val_acc 43,36% — lebih buruk dari baseline. Model besar (`capacity_deep`, 4 hidden layer) mencapai val_acc 49,44% — sedikit lebih baik dari baseline, tapi jauh di bawah `reg_batchnorm`. Trade-off waktu: `capacity_deep` early-stop di epoch 17 (±292 detik), menandakan model besar ini cepat berhenti belajar tanpa memberi lonjakan akurasi yang sepadan dengan tambahan parameter dan waktu komputasinya.

5. Rekomendasi Akhir Kombinasi paling efisien adalah arsitektur baseline (3072→512→256→10) + Batch Normalization, tanpa Dropout, tanpa mengubah optimizer/scheduler dari Adam default. Strategi ini mencapai akurasi tertinggi (58,68% validasi, 58,43% test) dengan gap train-val-test yang kecil, membuktikan generalisasi baik tanpa perlu menambah kompleksitas model atau regularisasi tambahan yang justru merugikan (seperti Dropout sendirian).

Catatan dari Confusion Matrix

Kesalahan klasifikasi terbesar terjadi antara:

- **Cat vs Dog** (189 cat diprediksi dog, 183 dog diprediksi cat)
- **Automobile vs Truck** (165 automobile diprediksi truck, 141 truck diprediksi automobile)

Hal ini masuk akal karena kemiripan bentuk/pose objek pada resolusi rendah 32×32 piksel, terutama untuk pasangan kelas yang secara visual memang mirip satu sama lain.